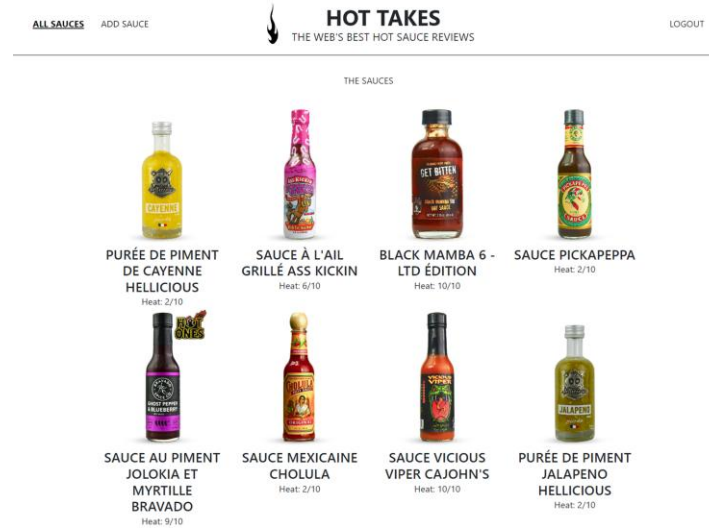


Construisez une API sécurisée pour une application d'avis gastronomiques

HOT TAKES



Contexte du projet

- Piquante se dédie à la création de sauces épicées dont les recettes sont gardées secrètes.
- Pour tirer parti de son succès et générer davantage de buzz, l'entreprise souhaite créer une application web dans laquelle les utilisateurs peuvent ajouter leurs sauces préférées
- et liker ou disliker les sauces ajoutées par les autres.

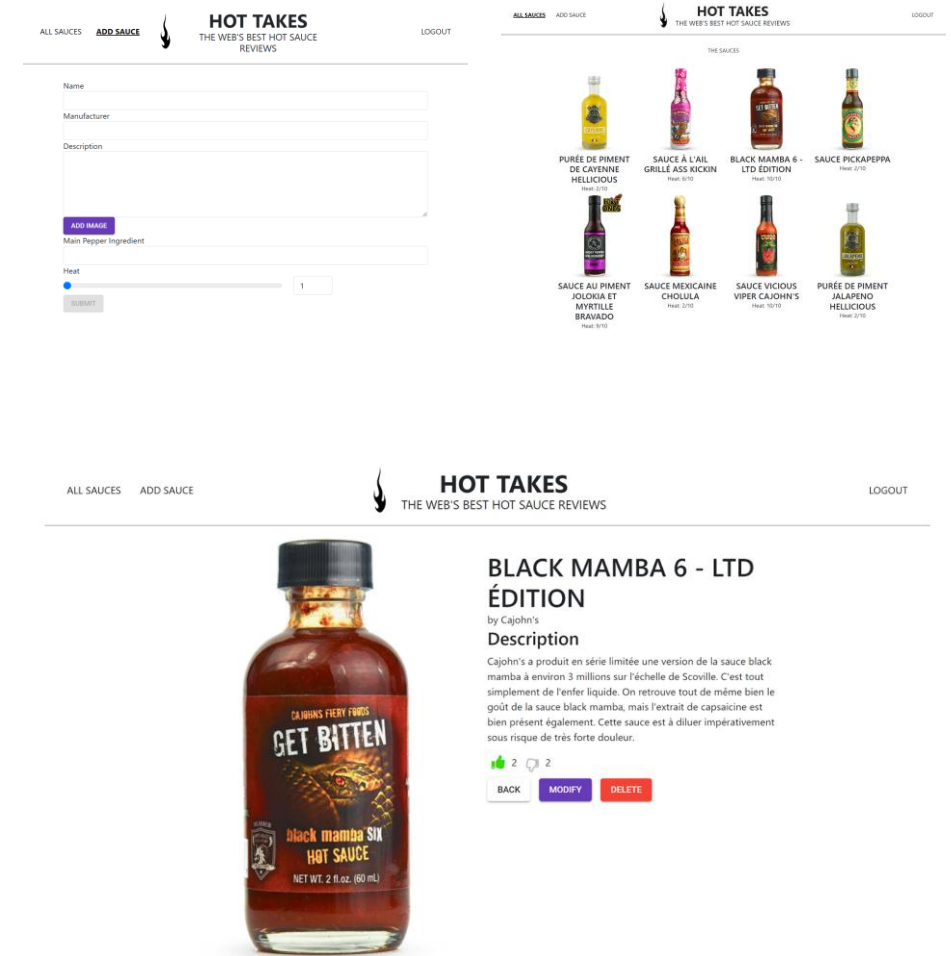


HOT TAKES

THE WEB'S BEST HOT SAUCE REVIEWS

Spécifications du projet

- une « galerie de sauces » permettant aux utilisateurs
 - de télécharger leurs sauces piquantes préférées et
 - de liker ou disliker les sauces que d'autres partagent.
- Le front-end de l'application a été développé à l'aide d'Angular et a été précompilé après des tests internes,
- mais Piquante a besoin d'un développeur back-end pour construire l'API.



Hot Takes



**PRÉSENTATION ARCHITECTURE
ET CHOIX TECHNOS**

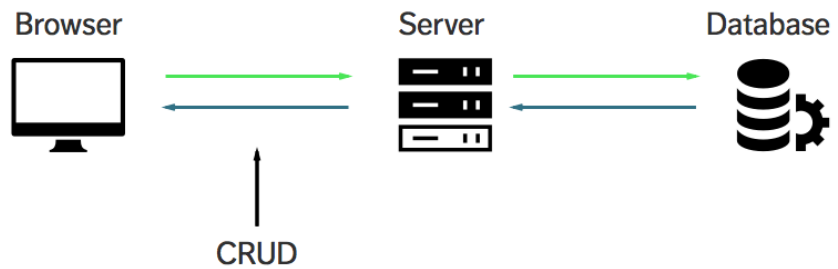


PRÉSENTATION DU CODE



SÉCURISATION DE L'API

Architecture

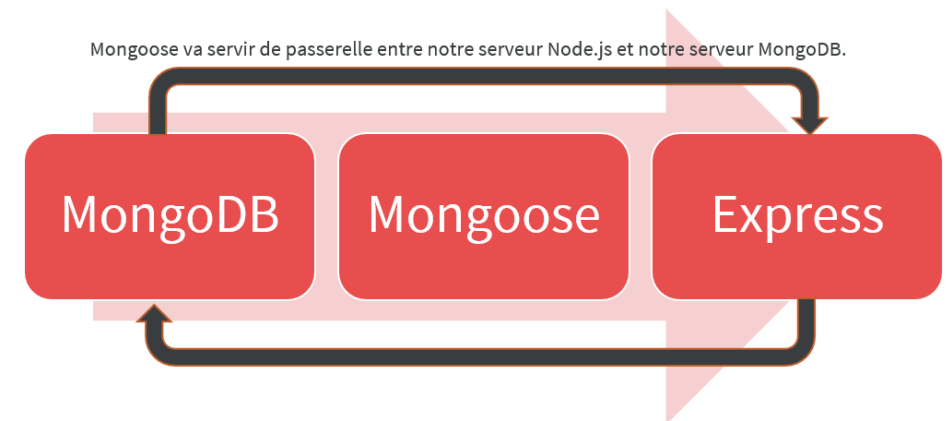


Techno & dépendances



- Le back-end repose sur la techno open source de Node
 - ✓ Programme haute performance optimisé pour tout types d'environnements
 - ✓ Ses packages s'installent facilement
 - ✓ MongoDB est compatible
 - ✓ Express ;
 - framework reposant sur Node,
 - facilite la création et la gestion des serveurs Node
 - ✓ Mongoose,
 - Mongoose est un package qui facilite les interactions avec notre base de données MongoDB
 - La méthode Schema de Mongoose vous permet de créer un schéma de données pour votre base de données MongoDB.

```
backend > {} package.json > {} dependencies
1
2 {
3   "name": "backend",
4   "version": "1.0.0",
5   "description": "",
6   "main": "server.js",
7   > Debug
8   "scripts": {
9     "test": "echo \"Error: no test specified\" && exit 1"
10  },
11  "author": "",
12  "license": "ISC",
13  "dependencies": {
14    "bcrypt": "^5.1.0",
15    "body-parser": "^1.20.1",
16    "cors": "^2.8.5",
17    "dotenv": "^16.0.3",
18    "express": "^4.18.2",
19    "express-rate-limit": "^6.7.0",
20    "helmet": "^6.0.1",
21    "jsonwebtoken": "^9.0.0",
22    "mongoose": "^6.8.1",
23    "mongoose-unique-validator": "^3.1.0",
24    "multer": "^1.4.5-lts.1",
25    "nodemon": "^2.0.20",
26    "password-validator": "^5.3.0",
27    "xss-clean": "^0.1.1"
28  }
29 }
```

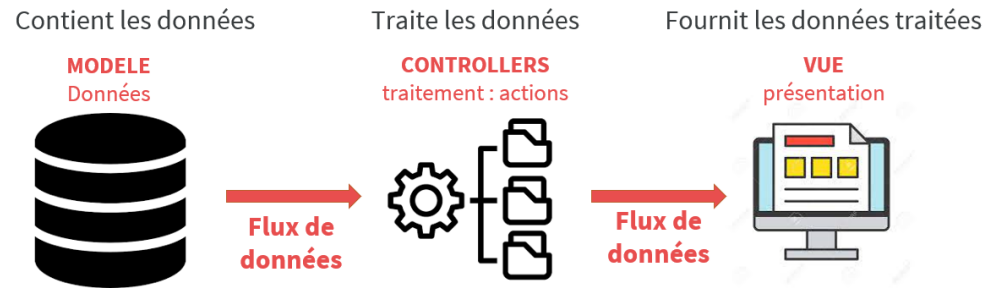


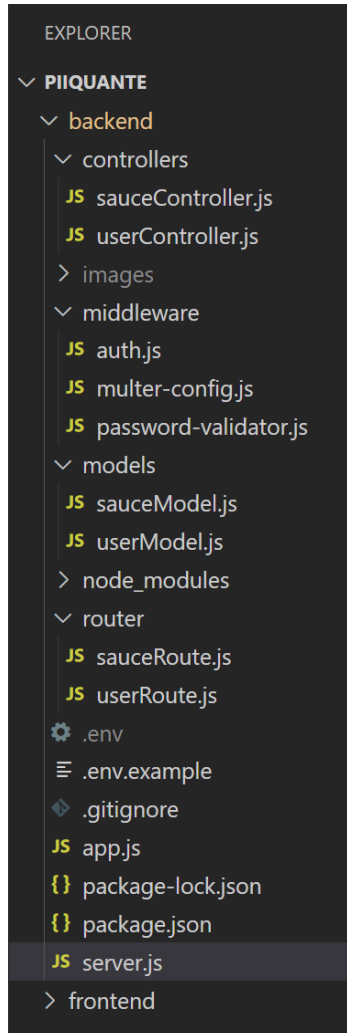
Architecture MVC

MVC est une façon d'organiser une interface graphique d'un programme. (motif de conception “design pattern”) il mène à une organisation rigoureuse et logique du code

- définition de règles permettant de savoir où ajouter une fonctionnalité
- simplification de la maintenance et de l'évolutivité de chacune
- clarté indispensable pour les gros projets : gain de temps
- il est très répandu et résulte de la formalisation de bonnes pratiques

Elle consiste à distinguer trois entités distinctes qui sont, le modèle, la vue et le contrôleur ayant chacun un rôle précis dans l'interface





Structure du code

1. App
 - Centralise toutes les infos
2. Routes
 - Définit les règles CRUD : les chemins d'accès aux dossiers
3. Models
 - Définit les schémas de la BDD
4. Controllers
 - C'est le code qui définit les actions
 - Instructions du back-end
5. Middleware
 - Les autorisations et gestion des images

Présentation du code



Modèle Logique métier

```
// Import du package mongoose pour la BDD
const mongoose = require('mongoose');
```

```
// Importation du package mongoose-unique-validator : améliore
const uniqueValidator = require('mongoose-unique-validator');
```

- Le modèle contient les données manipulées par le programme.
 - implante les fonctionnalités de l'application, indépendamment des aspects interactifs. (les règles, contraintes et traitements qui s'appliquent à ces données)
 - offre des méthodes pour mettre à jour ces données (insertion suppression, changement de valeur) ou pour récupérer ses données.
 - On utilise la méthode Schema mise à disposition par Mongoose pour créer notre modèle user et sauce
- **sauceModel.js ->**
 - Importation du package mongoose pour la DB et définition du chemin pour mongoose sous format JSON
- **userModel.js ->**
 - mongoose-unique-validator passé comme plug-in, s'assurera que deux utilisateurs ne puissent partager la même adresse e-mail.

```
//Création schéma de données user
const userSchema = mongoose.Schema({
  email: { type: String, required: true, unique: true},
  password:{ type: String, required: true},
});
```

```
//Création du schéma de données sauce
const sauceSchema = mongoose.Schema({
  userId: { type: String, required: true },
  name : { type: String, required: true },
  manufacturer : { type: String, required: true },
  description: { type: String, required: true },
  mainPepper : { type: String, required: true },
  imageUrl: { type: String, required: true },
  heat : { type: Number, required: true },
  likes : { type: Number, required: true },
  dislikes : { type: Number, required: true },
  usersLiked : [ String, "<userId>" ],
  usersDisliked : [ String, "<userId>" ]
});
```



Vue l'interface avec l'utilisateur



- Afficher les données qu'elle a récupérées auprès du modèle.
- Reçoit toutes les actions de l'utilisateur (clic de souris, boutons, ...).
- Ses différents événements sont envoyés au contrôleur.
 - si on intègre beaucoup de code de traitement, on empiète sur la logique métier
 - ne pas trop surcharger la Vue

userRoute.js & sauceRoute.js

• userRoute

- Importation du module express.router
- Création du chemin user dans controllers
- Les routes sign et login sont en mode POST
- Export du router

```
// Importer le module "router" de Express
const router = express.Router();
```

```
// Créer chemin user et sauce dans dossier controllers
const userController = require('../controllers/userController');
```

```
// Définir les router Sign-in et Login en méthode POST
router.post('/signup', passwordValidator, userController.signup);
router.post('/login', userController.login);
```

```
// Export pour pouvoir l'utiliser
module.exports = router;
```

• SauceRoute

- Définition des chemins sauces + authentification + multer qui serviront pour le router
- Définition pour chaque route de son chemin et ses droits
 - Chaque route a ses verbes d'actions / CRUD avec son chemin et ses droits

```
// Définir les chemins sauces / autorisation / et Multer qui serviront pour le/les router
const auth = require('../middleware/auth');
// multer permet de gérer les fichiers entrants dans les requêtes HTTP
const multer = require('../middleware/multer-config');
const sauceController = require('../controllers/sauceController');
```

```
// Définir les router avec CRUD en indiquant les chemins et les droits associés (Create/post Read/get Update/put Delete/delete)
router.get('/', auth, sauceController.getSauces);
router.post('/', auth, multer, sauceController.createSauces);
router.post('/:id/like', auth, sauceController.manageLike);
router.get('/:id', auth, sauceController.getOneSauce);
router.put('/:id', auth, multer, sauceController.updateSauce);
router.delete('/:id', auth, sauceController.deleteSauce);
```



Contrôleurs synchronisation du modèle et de la vue

- 
- Il reçoit tous les événements de l'utilisateur et enclenche les actions à effectuer.
 - Ils permettent de
 - structurer hiérarchiquement l'application
 - faciliter la compréhension du code et la maintenance
- ♠ Comme pour la Vue, on évitera de placer de logique applicative dans un contrôleur, car le code n'est pas réutilisable dans un autre contexte

sauceController.js

- sauceController.js
 - Appel de toutes les sauces avec « request » « result »
 - « next » pour passer au middleware/controller suivant
 - Demande à la BDD les sauces avec find()
 - Utiliser .then pour retour positif 200
 - Utiliser .catch pour récupérer erreur 500
 - Img et textes doivent être traités différemment elles sont appelées via leur url
 - Comme pour une url, il faut définir le chemin avec le protocole (http), hôte (localhost:4200/sauces) et le nom du fichier (filename)
 - Pour la suppression dans img, on utilise split()[1] et fs unlink

```
exports.getSauces = (req, res, next) => {  
  Sauce.find() // demande à BDD  
  .then((sauces) => res.status(200).json(sauces))  
  .catch((error) => res.status(500).json({error}))  
}
```

```
exports.createSauces = (req, res, next) => {  
  const sauceObj = JSON.parse(req.body.sauce);  
  delete req.body._id;  
  const sauce = new Sauce({  
    ...sauceObj,  
    imageUrl: `${req.protocol}://${req.get('host')}/images/${req.file.filename}`,  
    likes: 0,  
    dislikes: 0  
  });  
  console.log(sauce);  
  
  sauce.save()  
  .then(() => res.status(201).json({ message: 'Objet enregistré !'}))  
  .catch(error => res.status(400).json({ error }));  
}
```

```
exports.updateSauce = (req, res, next) => {  
  const sauceObject = req.file ? {  
    ...JSON.parse(req.body.sauce),  
    imageUrl: `${req.protocol}://${req.get('host')}/images/${req.file.filename}`  
  } : { ...req.body };  
  
  delete sauceObject._userId;  
  Sauce.findOne({ _id: req.params.id })  
  .then((sauce) => {  
    if (sauce.userId !== req.auth.userId) {  
      res.status(401).json({ message: 'Not authorized' });  
    } else {  
      Sauce.updateOne({ _id: req.params.id }, { ...sauceObject, _id: req.params.id })  
        .then(() => res.status(200).json({ message: 'Sauce modifiée !'}))  
        .catch(error => res.status(401).json({ error }));  
    }  
  })  
  .catch(error => {  
    res.status(400).json({ error });  
  });  
}
```

```
exports.deleteSauce = (req, res, next) => {  
  Sauce.findOne({ _id: req.params.id })  
  .then(sauce => {  
    if (sauce.userId !== req.auth.userId) {  
      res.status(401).json({ message: 'Not authorized' });  
    } else {  
      const filename = sauce.imageUrl.split('/images/')[1];  
      fs.unlink('images/' + filename, () => {  
        Sauce.deleteOne({ _id: req.params.id })  
          .then(() => {  
            res.status(200).json({ message: 'Sauce supprimée !' });  
          })  
          .catch(error => res.status(401).json({ error }));  
      });  
    }  
  })  
  .catch(error => {  
    res.status(500).json({ error });  
  });  
}
```

```

exports.managelike = (req, res) => {
  //on recupere la sauce sur laquelle modifier les likes
  Sauce.findOne({_id: req.params.id})
    .then((sauce) => {

      //l'utilisateur like la sauce : si la valeur de like est +1 on ajoute l'id du user connect
      if(req.body.like==1){
        sauce.usersLiked.push(req.auth.userId);
      }

      //l'utilisateur dislike la sauce : si la valeur de like est -1 on ajoute l'id du user connect
      else if(req.body.like==-1){
        sauce.usersDisliked.push(req.auth.userId);
      }

      //si la valeur de like est 0 on supprime l'id du user connecté des deux tableaux
      else if(req.body.like==0){
        //pour supprimer le user on filtre le tableau pour ne garder que les valeurs differentes
        if (sauce.userLiked.includes(req.body.userId)) {
          const userLikedIndex = sauce.userLiked.indexOf(req.body.userId);
          sauce.userLiked.splice(userLikedIndex, 1)
        }

        if (sauce.usersDisliked.includes(req.body.userId)) {
          const usersDislikedIndex = sauce.usersDisliked.indexOf(req.body.userId);
          sauce.usersDisliked.splice(usersDislikedIndex, 1)
        }
      }

      //on met a jour les compteurs de like et dislikes en comptant le nb d'element dans chacun des
      sauce.likes=sauce.usersLiked.length;
      sauce.dislikes=sauce.usersDisliked.length;

      //on met a jour l'objet dans la base avec les champs qu'on a modifié
      Sauce.updateOne({ _id: req.params.id}, sauce)
        .then(() => res.status(200).json({message : 'Sauce modifiée!'}))
        .catch(error => res.status(401).json({ error }));

    })
    .catch((error) => {
      console.log(error);
      res.status(400).json({ error });
    });
};

```

sauceController.js : Likes & Dislikes

- Appel des likes avec « request » et « result »
 - Identification de la sauce avec findOne() + id
 - Utilisation d'une promise .then? Pour le dislike, utilisation de like -1 dans le tableau en fonction de son id et + 1 pour les likes
- Le user s'est trompé, le like est égal à 0
 - Suppression dans le tableau des likes en fonction de l'id
 - Sauvegarde dans les tableau des données
 - Réponse des promises
 - fonction dont le rôle est d'effectuer une opération asynchrone et cette fonction va, lors de son exécution, créer et renvoyer un objet Promesse.

Présentation code App.js

- Importation des packages de Node.js

```
// Importer package Express
const express = require('express');
```

```
// Importer package Body-parser
const bodyParser = require('body-parser');
```

```
// Importer path pour définir les chemins
const path = require('path');
```

```
// Importer package mongoose
const mongoose = require('mongoose');
```

- Connection à la BDD de MongoDB avec user id et mot de passe

```
// Se connecter à la base de données avec id User et mot de passe
mongoose.connect(`mongodb+srv://${process.env.MONGODB_USER}:${process.env.MONGODB_PASSWORD}@cluster0.73rmxkv.mongodb.net/piiquante?retryWrites=true&w=majority`,
{ useNewUrlParser: true,
  useUnifiedTopology: true })
.then(() => console.log('Connexion à MongoDB réussie !'))
.catch(() => console.log('Connexion à MongoDB échouée !'));
```

- CORS Headers

```
// CORS MIDDLEWARES
// Ajouter "CORS Headers" pour définir qui peut accéder à l'API
app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content, Accept, Content-Type, Authorization');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, PATCH, OPTIONS');
  res.setHeader('Cross-Origin-Resource-Policy', 'same-site');
  next();
});
```

- Gestion des chemins de l'API pour Sauces + auth + Images

```
// Appeler les routes créées précédemment : userRoute et sauceRoute
const userRoute = require('./router/userRoute');
const sauceRoute = require('./router/sauceRoute');
```


Création d'un utilisateur

- Mot de passe doit être « hashé »
 - nous appelons la fonction de hachage de bcrypt dans notre mot de passe et lui demandons de « saler » le mot de passe 10 fois (bcrypt est un package de cryptage installé avec npm)
 - La méthode `hash()` de bcrypt crée un hash crypté des mots de passe de vos utilisateurs pour les enregistrer de manière sécurisée dans la base de données.
 - nous créons un utilisateur et l'enregistrons dans la base de données
 - Jsonwebtoken pour de générer des tokens d'authentification chiffrés qui peuvent être utilisés pour l'autorisation.

```
// Authentification_Pour créer un nouvel utilisateur dans la base de données
exports.signup = (req, res, next) => {
  console.log (req.body);
  // nous appelons la fonction de hachage de bcrypt dans notre mot de passe
  bcrypt.hash(req.body.password, 10)
  // La méthode hash() de bcrypt crée un hash crypté des mots de passe
  .then((hashPassword) => {
    // nous créons un utilisateur et l'enregistrons dans la base de données
    const newUser = new User({
      email:req.body.email,
      password: hashPassword
    });
    newUser.save()
    .then(() => res.status(201).json({message: " utilisateur créé !"}))
    .catch((error) => res.status(500).json ({error}))
  })
  .catch((error) => res.status(500).json ({error}))
}
```

Authentification

- Middleware authentification
 - Configurer clé d'authentification
- Prérequis mot de passe
 - password-validator

```
// Configure Auth Key
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  try {
    const token = req.headers.authorization.split(' ')[1];
    const decodedToken = jwt.verify(token, process.env.TOKEN);
    const userId = decodedToken.userId;
    req.auth = {
      userId: userId
    };
    next();
  } catch(error) {
    res.status(401).json({ error });
  }
};
```

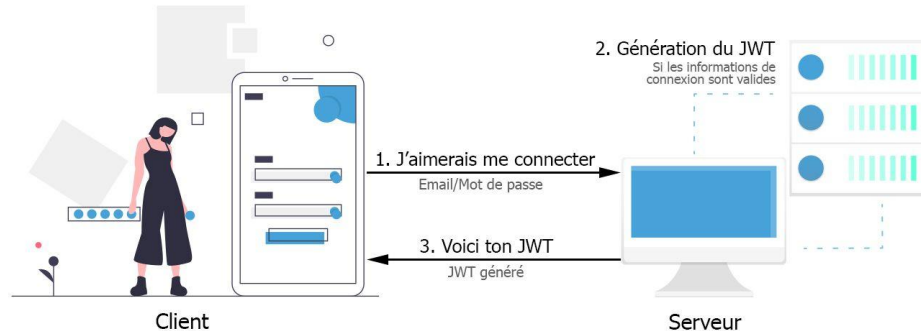
```
//
const passwordValidator = require('password-validator');

// Create a schema
const schema = new passwordValidator();

// Add properties to it
schema
  .is().min(8) // Minimum length 8
  .is().max(100) // Maximum length 100
  .has().uppercase() // Must have uppercase letters
  .has().lowercase() // Must have lowercase letters
  .has().digits(2) // Must have at least 2 digits
  .has().not().spaces() // Should not have spaces
  .is().not().oneOf(['Password', 'Password123']); // Blacklist these values

module.exports = (req, res, next) => {
  if (schema.validate(req.body.password)){
    next();
  } else {
    return res.status(400).json({message : "mot de passe faible"});
  }
};
```

Se connecter et disposer d'un token valide



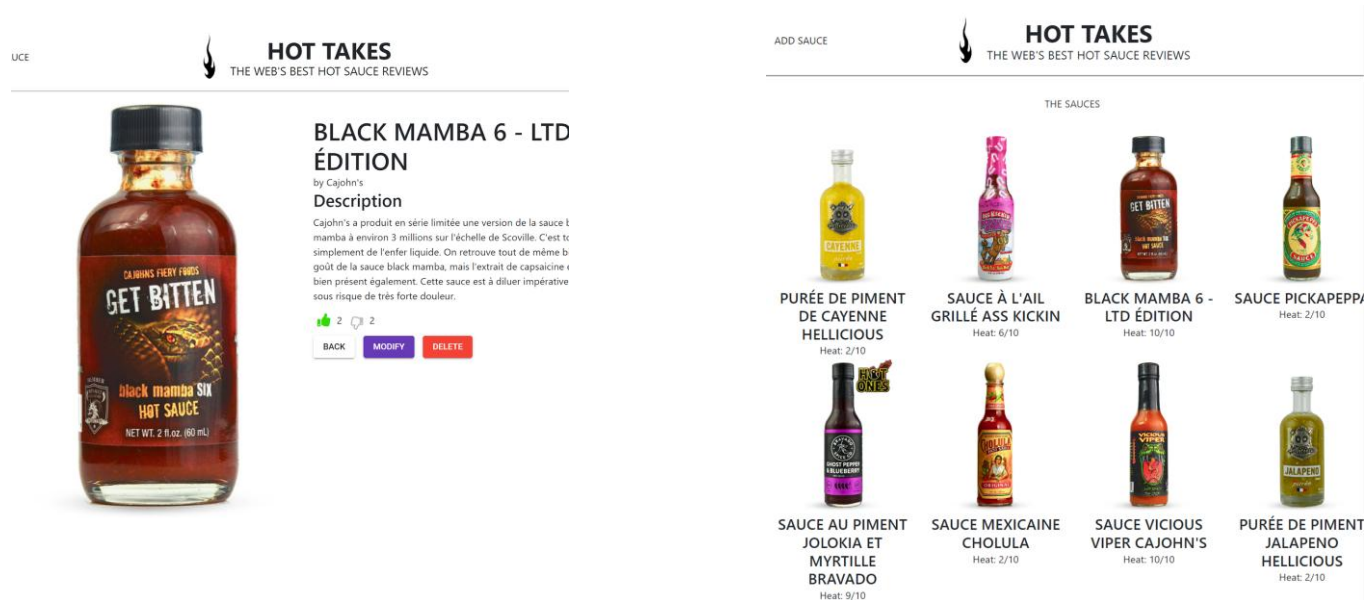
- Rechercher si l'utilisateur est enregistré dans la base de données user :
 - en utilisant la méthode `findOne()` dans notre modèle
- Utiliser la fonction `compare` de `bcrypt` pour comparer le mot de passe entré par l'utilisateur avec le hash enregistré dans la BDD.
- S'ils ne correspondent pas, nous renvoyons le même message que lorsque l'utilisateur n'a pas été trouvé

```
// Authentication_Pour connecter un utilisateur
exports.login = (req, res, next) => {
  // Rechercher si l'utilisateur est enregistré dans la base de données USER en utilisant la méthode findOne() de mongoose
  // find a User by email: findOne({ where: { email: ... } })
  User.findOne({ email: req.body.email })
    .then(user => {
      if (!user) {
        return res.status(401).json({ error: 'Paire identifiant et mot de passe incorrecte' });
      }
      // Utiliser la fonction compare de bcrypt pour comparer le mot de passe entré par l'utilisateur avec le hash enregistré
      bcrypt.compare(req.body.password, user.password)
        .then(valid => {
          // S'ils ne correspondent pas, nous renvoyons une erreur 401 Unauthorized avec le même message que lorsque l'utilisateur n'a pas été trouvé
          if (!valid) {
            return res.status(401).json({ error: 'Paire identifiant et mot de passe incorrecte !' });
          }
          // S'ils correspondent, nous renvoyons l'ID de l'utilisateur et un token.
          res.status(200).json({
            userId: user._id,
            // Nous utilisons la fonction sign de jsonwebtoken pour chiffrer un nouveau token.
            token: jwt.sign(
              // Ce token contient l'ID de l'utilisateur en tant que payload (les données encodées dans le token)
              { userId: user._id },
              process.env.TOKEN,
              // Nous renvoyons le token au front-end avec notre réponse.
              // Nous définissons la durée de validité du token à 24 heures.
              { expiresIn: '24h' }
            )
          });
        })
        .catch(error => res.status(500).json({ error }));
    })
    .catch(error => res.status(500).json({ error }));
};
```

Ajout d'une sauce

- Middleware multer-config.js
- Appel de multer : Définition des types de MIMES acceptés
 - Destination du stockage des images
 - création d'un nom de fichier image unique et remplacement par des « _ »
- Nous exportons ensuite l'élément multer entièrement configuré, en lui passons
 - notre constante storage
 - et en lui indiquant que nous gérerons uniquement les téléchargements de fichiers image.

```
backend > middleware > JS multer-config.js > ...
1 // Appel du plugin multer
2 const multer = require('multer');
3
4 // Définition des types de MIMES acceptés
5 const MIME_TYPES = {
6   'image/jpg': 'jpg',
7   'image/jpeg': 'jpg',
8   'image/png': 'png'
9 };
10
11 // Destination du stockage des images
12 const storage = multer.diskStorage({
13   destination: (req, file, callback) => {
14     callback(null, 'images');
15   },
16   filename: (req, file, callback) => {
17     // création d'un nom de fichier image unique et remplacement
18     const name = file.originalname.split(' ').join('_');
19     const extension = MIME_TYPES[file.mimetype];
20     callback(null, name + Date.now() + '.' + extension)
21   }
22 });
23
24 // Export de l'élément multer entièrement configuré (de
25 module.exports = multer({storage: storage}).single('image');
```



Liker ou disliker sauces

Propres sauces et sauces ajoutées par d'autres users



SAUCE VICIOUS VIPER CAJOHN'S

by Cajohn's

Description

La sauce vicious viper est une sauce à base de piment habanero, de fruits exotiques et d'extrait de capsaine. On apprécie particulièrement l'équilibre de l'ensemble. Cependant, avec 250 000 unités sur l'échelle de Scoville cette sauce se déguste avec précaution. A ne pas donner aux enfants.

👍 0 🗨️ 2

BACK



PURÉE DE PIMENT DE CAYENNE

by Hellicious

Description

La purée de piment de Cayenne jaune est une fabrication artisanale par Sauce Piquante, on y a ajouté un peu de vinaigre pour la conservation. On aime utiliser le piment de Cayenne avec de la tomate, du poivron ou de la courgette. On l'utilise aussi pour ajouter quelques gouttes dans nos salsas maison.

👍 2 🗨️ 1

BACK



PURÉE DE PIMENT DE CAYENNE

by Hellicious

Description

La purée de piment de Cayenne jaune est une fabrication artisanale par Sauce Piquante, on y a ajouté un peu de vinaigre pour la conservation. On aime utiliser le piment de Cayenne avec de la tomate, du poivron ou de la courgette. On l'utilise aussi pour ajouter quelques gouttes dans nos salsas maison.

👍 2 🗨️ 1

BACK

MODIFY

DELETE

Sécurisation

RGPD & OWASP





Sécurité Utilisateur

- Piratage de session bloqué
 - Utilisation d'un token unique avec une clef d'identification grâce au plugin jsonwebtoken
- Protection des données sur l'app
 - Utilisation du plugin bcrypt afin de hasher le mot de passe

```
backend >  .env.example
1 MONGODB_PASSWORD =
2 MONGODB_USER =
3 TOKEN =
```

```
backend > .env
1 MONGODB_PASSWORD = cobain
2 MONGODB_USER = chainaa
3 TOKEN = 'c4b25220-8b87-11ed-b1ff-35d0e857da2'
```

```
// dotenv -> Variables d'environnement
const dotEnv = require('dotEnv').config('./env');
```

```

# Configure MongoDB
mongoos.connect('mongodb://[4]process.env.MONGO_DB_USER:[5]process.env.MONGO_DB_PASSWORD@[6]listenB37rmaks.mongoos.net:27020/?authSource=admin&majority=1', {
  useUnifiedTopology: true,
  useNewUrlParser: true
})
.then(() => console.log('Connexion à MongoDB réussie !'))
.catch(() => console.log('Connexion à MongoDB échouée !'))

```

Sécurisation de l'accès à la base de données

- Accès à la base de données et token dissimulés grâce à dotenv et aux Variables d'environnement
 - dotenv est un module sans dépendance qui charge nos variables à partir du fichier .env.



sécurité

- Serveur
 - 1ere barrière contre les hackers avec les firewalls
- Node.js
 - Permet d'avoir une api organisée avec plusieurs dossiers
 - Aucun risque d'accéder à tout le contenu grâce aux plugin
- mongoDB
 - Bdd dans un cloud
 - Les infos sont externalisées et même lors d'une attaque, le hacker n'aura pas accès aux données
 - Très peu d'injections possible, puisque mongoDB n'utilise pas la techno SQL



XSS
Cross Site
Scripting



Sécurisation Routes Node

- Helmet
 - Ce module très célèbre est en fait un regroupement de plusieurs petits modules qui permettront de sécuriser votre HTTP headers.
- XSS
 - Une attaque XSS, c'est l'injection d'un script dans votre serveur, ce qui modifiera son comportement.
 - Ces attaques sont connues dans le monde du dev et plusieurs solutions existent déjà. La plus simple pour votre application node est d'installer un module qui va gérer pour vous. Un des plus connu, c'est XSS.
- express-rate-limit
 - Si une personne veut faire tomber un serveur, il peut envoyer des requêtes en boucle. Ce qui aura comme conséquence de faire stresser votre application Node.
 - Un moyen pour empêcher ce genre de pratique est de limiter le nombre de requête que peut faire un client. Le module que j'utilise personnellement est express-rate-limit



Sécurisation du stockage

- mongoDB
 - Bdd dans un cloud
 - Les infos sont externalisées et même lors d'une attaque, le hacker n'aura pas accès aux données
 - Très peu d'injections possible, puisque mongoDB n'utilise pas la techno SQL
- dotenv
 - module sans dépendance qui charge nos variables à partir du fichier .env.



Merci

Q&R

